Quiz 1: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 1 is Thursday Sep 3, at the end of class. ~12 minutes, 10 marks.

Covers: the Week 1 lecture (Aug 27) and **Lab 1**.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed.

A formula sheet is handed out with the quiz (`formula-sheet.md`), so nothing needs memorizing. Read it now rather than on Thursday: it lists **more formulas than any one question needs**, so deciding which applies is still your job.

The goal is not memorization. It is demonstrating that you understand the six components and can apply them to a small problem by hand.

How to use this guide

Roughly half the quiz is one small worked problem: a tiny dataset, a linear model, a loss, and a single step of gradient descent. Roughly a third asks you to **explain what Lab 1 demonstrated**, which is why running it yourself matters. The rest is short conceptual answers.

No question asks you to recall a number from your lab. The lab questions describe something the notebook put on your screen and ask *why* it happened, so the way to prepare is to scroll back through your own plots until you can account for each.

If you can do everything in Part 1 below on paper without notes, and you can explain the seven observations listed in Part 8, you are ready.

Part 1: The six components

Every model in this course is the same six pieces. Be able to **name all six** and say what each one does.

#	Component	What it is
1	**Data**	Inputs $\mathbf{x}$ and targets y
2	**Parametric model**	$\hat{y} = m(\mathbf{x} \mid \mathbf{w})$, a function with tunable parameters $\mathbf{w}$
3	**Residuals**	$y - \hat{y}$, how wrong each prediction is
4	**Loss**	One number scoring a whole parameterization, $\mathcal{L}(\mathbf{w})$
5	**Splits**	Train / validation / test
6	**Optimizer**	The procedure that searches for better $\mathbf{w}$

Training = optimization. The goal is always to find parameters $\mathbf{w}$ that make the loss small.

Part 2: The parametric model

Linear regression

$$\hat{y} = w_0 + w_1 x$$

- w_1 is the slope (weight), w_0 is the intercept (**bias**).
- $\mathbf{w} = (w_0, w_1)$ is the **parameter vector**. One point in parameter space is one specific model.
- More parameters means more flexibility.

Two spaces to keep straight

This distinction shows up on the quiz, so be deliberate about it.

- **Data space**: axes are x and y. A *model* is a **line** in this space.
- **Parameter space**: axes are w_0 and w_1. A *model* is a **point** in this space, and training is a **path** through it.

Drawing it as a network

You should be able to draw $\hat{y} = w_0 + w_1 x$ as a network diagram:

- one **input node** labeled x
- one **output node** labeled $\hat{y}$
- the **edge** between them labeled with the weight w_1
- the **bias** w_0 marked on the output node

Logistic regression is the same picture with a sigmoid applied at the output:

$$\hat{p} = \sigma(w_0 + w_1 x_1 + w_2 x_2), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

This is the hinge of the whole lecture. Linear and logistic regression, which you already know, are one-neuron networks.

Part 3: Residuals, loss, and metrics

Losses you should recognize

Mean squared error (MSE), for regression. Punishes large errors hard, because the error is squared.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Mean absolute error (MAE), for regression. More tolerant of outliers than MSE, because the error is not squared.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Binary cross-entropy (BCE), for binary classification. Takes predicted probabilities, not raw scores.

$$\text{BCE} = -\frac{1}{n}\sum_{i=1}^{n}\left[\, y_i \log \hat{p}_i + (1 - y_i)\log(1 - \hat{p}_i)\,\right]$$

Know how to **compute MSE by hand** on three or four points: predict, subtract, square, average. Note the **mean**: dividing by n is part of the definition.

Loss vs cost. Strictly, the *loss* is per example and the *cost* is the single number averaged over the dataset. The terms are used interchangeably in practice, but know the distinction.

Why BCE punishes confident wrong answers: as $\hat{p} \to 0$ while the true label is 1, $-\log \hat{p} \to \infty$. Being confidently wrong is catastrophically expensive; being uncertain is merely mildly bad.

You do not need to memorize a catalog of losses. There are dozens. You need to know which family your problem calls for, and how to recognize when something has chosen the wrong one for you.

Metrics are not losses

- **Loss**: what the optimizer minimizes. **Must be differentiable.**
- **Metric**: what you actually care about and report to humans. Accuracy, F1, precision at 90% recall, dollars saved.

These are frequently **not the same function**. Accuracy is **not differentiable**, so you cannot train on it: it is flat almost everywhere with jumps at the decision threshold, so the gradient carries no information. You train on cross-entropy and *report* accuracy.

Also know that a metric is meaningless without a **baseline**. If 79% of your labels are class 1, then 79% accuracy is what you get for predicting class 1 every time. Compare to the majority-class baseline before being pleased.

Part 4: Splits and overfitting

Split	Purpose
Train	The model learns from this
Validation	You tune *your* choices against this (capacity, learning rate, when to stop)
Test	Touched **once**, at the very end, to report a number

You cannot evaluate a model on the data it learned from; it has already seen the answers. And once you tune against the test set, it stops being a test set and becomes a second validation set, so your reported number is optimistic.

The most important plot in machine learning

Loss on the vertical axis, training iteration (or model capacity) on the horizontal, with two curves:

- **Training loss** keeps going down: the model keeps memorizing.
- **Validation loss** bottoms out, then **turns back up**.
- That turning point is where the model stopped learning the pattern and started learning the noise.

Early stopping is the simplest fix: stop at the bottom.

Be able to sketch this, label the underfitting region, the optimal region, the overfitting region, and mark where early stopping would trigger.

Why training loss can never choose capacity

Adding parameters can only help the model fit the points it trained on, so training loss falls monotonically with capacity and has **no interior minimum**. It cannot distinguish signal from noise. Only held-out data can.

The noise floor (irreducible error)

If targets are generated as a true function **plus independent random noise**, that noise is unrelated to the inputs and therefore **unpredictable by any model**. Its variance is a hard floor on achievable loss. Lab 1 makes this concrete: the noise standard deviation is 0.25, so the floor is $0.25^2 = 0.0625$, and your test MSE should land **near** it.

Near, in both directions. The floor is an *expected* value over the whole population, while your test MSE is an estimate of it from only 80 test points. So it scatters around the floor, and it is completely normal for it to come out a little **below** 0.0625. That is sampling luck, not a model that beat the irreducible error. Do not report that you beat the noise floor.

Part 5: Optimization

Gradients, only as much as you need

This is the entire calculus requirement for the quiz.

- A **partial derivative** $\dfrac{\partial \mathcal{L}}{\partial w_i}$ is the slope of the loss if you nudge **one** parameter and hold the rest still.
- The **gradient** is the vector of all of them:

$$\nabla \mathcal{L}(\mathbf{w}) = \left(\frac{\partial \mathcal{L}}{\partial w_0}, \frac{\partial \mathcal{L}}{\partial w_1}, \ldots, \frac{\partial \mathcal{L}}{\partial w_N} \right)$$

- It points in the direction of **steepest increase**. So $-\nabla \mathcal{L}$ points **downhill**.
- At the bottom of a valley the surface is flat: $\nabla \mathcal{L}(\mathbf{w}) = 0$.

You will **not** be asked to differentiate a complicated function symbolically. You will be given a gradient and asked to use it.

Gradient descent

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \lambda \nabla \mathcal{L}(\mathbf{w}_{\text{old}})$$

1. Look at the slope where you are standing.
2. Take a step downhill.
3. Repeat.

Be able to **carry out one step by hand** given $\mathbf{w}_{\text{old}}$, a gradient, and a learning rate. Mind the **sign**: you *subtract*. Adding the gradient walks uphill and increases the loss, which is the single most common error on this question.

For a single parameter the same rule is $w^j_{\text{new}} = w^j_{\text{old}} - \lambda \frac{\partial \mathcal{L}}{\partial w^j}$.

The learning rate λ

λ controls **how big a step** you take.

- **Too small**: you never arrive in the iterations you have.
- **Too large**: you overshoot the valley and the loss *increases*, sometimes diverging entirely.

In Lab 1 you saw all three behaviors from the same code.

The loss surface

For a two-parameter model, $\mathcal{L}(w_0, w_1)$ is a **surface** over parameter space, and gradient descent is a path downhill across it. For MSE with a linear model the surface is a convex bowl (a paraboloid) with one minimum. In general, for a neural network, it is **not** convex: there is more than one place to stop, and where you land depends on where you started. That is why Week 4 exists.

Part 6: Where deep learning sits, and problem types

You should be able to place these correctly:

- **AI** $\supset$ **Machine learning** $\supset$ **Deep learning**. They are not synonyms.
- **Traditional programming**: data + rules $\rightarrow$ answers. **Machine learning**: data + answers $\rightarrow$ rules.
- **Paradigms**: supervised, unsupervised, semi-supervised, self-supervised, reinforcement learning.
- **Supervised** splits into **classification** (discrete target) and **regression** (continuous target).

Know the four problem types and their input $\rightarrow$ output shapes:

Type	Output	Typical final activation	Typical loss
Scalar regression	one number	none	MSE / MAE
Vector regression	several numbers	none	MSE / MAE
Binary classification	one probability	sigmoid	binary cross-entropy
Multi-class classification	probability per class	softmax	categorical cross-entropy

---

Part 7: What makes it a neural network

Two moves. That is the entire idea.

1. **Stack them.** Take many one-neuron units and arrange them in **layers**, feeding forward.
2. **Put a nonlinearity between the layers.**

Why the second move is not optional: without it, stacked linear layers **collapse back into a single linear layer**. A composition of linear maps is linear, so you gain no expressive power from the depth at all. Be able to state this.

Everything else this semester (CNNs, transformers, diffusion models) is a clever variation on those two moves.

Part 8: From Lab 1

Three of the ten marks come from Lab 1, and none of them ask for a number. Everyone ran the same seed, so the questions state what happened and ask you to explain the mechanism behind it. Open your notebook, look at your own plots, and make sure you can say each of these in a sentence or two:

1. **Why the split has to come before any model is fit.** What is wrong with judging a model on the rows it learned from.
2. **Why MSE and MAE disagree about outliers**, and which you would choose if the extreme values were genuine versus if they were sensor glitches.
3. **What the learning rate controls, and both ways it fails.** You saw one rate crawl and one diverge. Why does too large a step make the loss go *up* rather than just converge slowly?
4. **Why training loss cannot pick the polynomial degree.** Training MSE fell at every degree from 1 to 20 while validation turned sharply upward. What was the degree-20 model doing to the 30 training points, and why was training loss structurally incapable of warning you?
5. **Why accuracy changed when the threshold moved from 0.50 to 0.70** even though the model was untouched, and why BCE rather than accuracy is what the optimizer minimizes.
6. **Why the majority-class baseline has to be reported** next to a classifier's accuracy before that accuracy means anything.
7. **What the noise floor is**, why no model could push test MSE meaningfully below it, and why a value slightly below the floor is sampling variation rather than a better model.

These are the same seven points listed at the end of the lab notebook.

Checklist

Before Thursday, confirm you can:

- ☐ Name the six components and say what each does
- ☐ Draw linear or logistic regression as a network diagram, bias included
- ☐ Compute MSE by hand on a handful of points (remember the mean)
- ☐ State the difference between a loss and a metric, and why accuracy fails as a loss
- ☐ Say what a partial derivative is, what the gradient is, and what holds at a minimum
- ☐ Perform one gradient descent step by hand, with the correct sign
- ☐ Say what the learning rate controls and both ways it fails
- ☐ Sketch and label the train/validation loss curve, and mark early stopping
- ☐ Explain why training loss cannot select model capacity
- ☐ Name the two moves that make a neural network, and what breaks without the second
- ☐ Explain all seven Lab 1 observations in Part 8, from your own plots
- ☐ Arrive with a calculator and nothing else; no notes are permitted